CSCI 49376 – Big Data Technology
Assignment 1
Relational Database Design and Normalization of Large Datasets

Dataset Selection

For this assignment, you will select **ONE** of the following datasets.

Option 1 – Revenue for Cab Drivers

Source: Kaggle

Dataset: Revenue for Cab Drivers

Kaggle Link:

https://www.kaggle.com/datasets/diishasiing/revenue-for-cab-drivers

This dataset contains transportation and taxi-related information stored in CSV format.

Download the CSV file before beginning the assignment.

Option 2 – Diabetes 130-US Hospitals for Years 1999–2008

Source: Kaggle

Dataset: Diabetes 130-US Hospitals for Years 1999–2008

Kaggle Link:

https://www.kaggle.com/datasets/brandao/diabetes

This dataset contains hospital encounter records for patients diagnosed with diabetes. The dataset includes information related to hospital encounters, patient characteristics, admissions, diagnoses, laboratory testing, medications, and readmission.

Ignore the following columns: metformin repaglinide nateglinide chlorpropamide glimepiride acetohexamide glipizide glyburide tolbutamide pioglitazone rosiglitazone acarbose miglitol troglitazone tolazamide examide citoglipton insulin glyburide-metformin glipizide-metformin glimepiride-pioglitazone metformin-rosiglitazone metformin-pioglitazone change

Download the `diabetic_data.csv` file before beginning the assignment.

Assignment Overview

The provided datasets contain information stored primarily in a large, relatively flat tabular structure.

For this assignment, you will select **one dataset** and redesign its data as a relational database.

You will:

1. Analyze the original dataset.
2. Identify attributes and dependencies.
3. Apply database normalization.
4. Identify appropriate entities and relationships.

5. Construct an Entity-Relationship (ER) diagram.
6. Convert the ER diagram into a relational schema.
7. Implement the database using SQL.
8. Import and distribute the original CSV data among the normalized tables.

The objective of this assignment is to demonstrate the complete process of transforming a large dataset into a properly structured relational database.

You are responsible for determining the appropriate database design. The dataset does not automatically determine what your final tables should be.

Part 1 – Select and Examine the Original Dataset

Select **ONE** of the two approved datasets.

Before designing your database, carefully examine the CSV file and understand its structure.

Identify the columns contained in the dataset and determine what type of information each column represents.

Your report must include:

1. Dataset selected.
2. Name of the CSV file used.
3. Total number of rows.
4. Total number of columns.
5. List of all attributes contained in the original CSV file.
6. Brief description of what each attribute represents.
7. Identification of possible identifiers or keys found in the original data.
8. Identification of missing, unknown, or incomplete values, if applicable.

Do not immediately begin creating SQL tables. You should first understand the organization and meaning of the original data.

Part 2 – Normalize the Dataset

The original dataset contains different types of information within a large table.

Your task is to determine how this information can be reorganized into a collection of smaller, logically related tables.

Apply the normalization process:

$$\boxed{\text{Original Relation} \rightarrow \text{1NF} \rightarrow \text{2NF}}$$

For each stage of normalization:

1. Show the structure of the relation or relations.
2. Identify the appropriate keys.
3. Identify relevant functional dependencies.

4. Identify redundancy or dependency problems.
5. Explain what changes are necessary to move to the next normal form.
6. Briefly explain why the resulting design satisfies the requirements of that normal form.

Your final database design must satisfy **Second Normal Form (2NF)**.

You are responsible for determining the appropriate entities and tables based on your analysis of your selected dataset.

For every table in your final design, clearly identify:

- Table name
- Attributes
- Primary key
- Foreign key(s), if applicable

Your design should minimize unnecessary data redundancy while preserving the information contained in the original dataset.

Part 3 – Construct an ER Diagram

Using the normalized database design developed in Part 2, construct a complete **Entity-Relationship (ER) diagram**.

Your ER diagram must clearly show:

- All entities
- Important attributes
- Primary keys
- Foreign keys where appropriate
- Relationships between entities
- Cardinalities
- Participation or relationship constraints where appropriate

Carefully determine whether each relationship is:

- One-to-One (1:1)
- One-to-Many (1:N)
- Many-to-Many (M:N)

Your ER diagram must be consistent with the normalization performed in Part 2.

You may use an appropriate database modeling or diagramming application to create the ER diagram.

Part 4 – Convert the ER Diagram into Relational Tables

Convert your completed ER diagram into a relational database schema.

For every relation, provide:

- Relation/table name

- Attributes
- Primary key
- Foreign key(s)

Clearly indicate how the relationships shown in your ER diagram are represented in the relational schema.

The relational schema developed in this section must be consistent with your ER diagram.

Part 5 – Implement the Database Using SQL

Implement your relational database using SQL.

Write a complete SQL script containing the necessary `CREATE TABLE` statements for all tables identified in your final database design.

Your SQL implementation should use appropriate:

- Table names
- Column names
- SQL data types
- Primary-key constraints
- Foreign-key constraints
- `NOT NULL` constraints where appropriate
- `UNIQUE` constraints where appropriate

Tables should be created in an appropriate order so that foreign-key relationships can be established correctly.

Your SQL implementation must correspond to the relational schema and ER diagram developed in the previous sections.

Part 6 – Import the CSV Data

After successfully creating all relational tables, populate the database using the data contained in your selected dataset.

You should not manually enter individual records from the dataset.

Develop an appropriate method for importing the CSV data and distributing the information among your normalized tables.

A temporary or **staging table** may be used to initially store the original CSV data.

The general process is:

CSV File → Staging Table → Normalized Tables

Your import procedure must:

1. Read or import the original CSV data.
2. Store the original data in an appropriate staging table if needed.
3. Identify the data required for each normalized table.

4. Insert the appropriate records into each table.
5. Avoid unnecessary duplicate records.
6. Preserve all primary-key relationships.
7. Preserve all foreign-key relationships.
8. Populate the final database using the information contained in the original dataset.

The following general SQL structure demonstrates how selected columns may be transferred from a staging table into another table:

```
INSERT INTO New_Table (
    new_column1,
    new_column2
)
SELECT
    original_column1,
    original_column2
FROM Staging_Table;
```

When appropriate, you may need to investigate techniques such as:

```
SELECT DISTINCT ...
FROM Staging_Table;
```

to prevent unnecessary duplicate records.

The examples above demonstrate only the general SQL mechanism.

You must determine which columns belong in each table based on your own normalization and ER design.

Document the procedure that you used to transfer the data from the original CSV file into the normalized database.

Part 7 – Verify the Database Using SQL

After importing the data, verify that your relational database was populated correctly.

Write SQL queries that demonstrate that:

- Tables contain data.
- Primary keys are correctly populated.
- Foreign-key values are correctly populated.
- The expected number of records were transferred into the appropriate tables.
- The normalized database preserves the necessary information from the original dataset.

Use verification queries such as:

```
SELECT *
FROM Table_Name;
```

and:

```
SELECT COUNT(*)
FROM Table_Name;
```

where appropriate.

You should provide screenshots or query results demonstrating that your database was successfully created and populated.

Required Deliverables

Submit a complete report documenting your database design and implementation.

Your submission must contain:

1. Name of the selected dataset.
2. Description of the original dataset.
3. List and description of the original attributes.
4. Original dataset dimensions (rows and columns).
5. Normalization process showing:
   - First Normal Form (1NF)
   - Second Normal Form (2NF)
6. Relevant functional dependencies.
7. Final list of entities and attributes.
8. Identification of all primary keys and foreign keys.
9. Complete ER diagram with cardinalities and participations.
10. Final relational schema.
11. SQL `CREATE TABLE` statements. Code should be in txt file.
12. SQL statements or scripts used to import and distribute the CSV data.
13. Screenshots demonstrating that the tables were successfully created and populated.
14. Everyone in the team will submit the same document and code.

Submission Organization

Organize your submission as follows:

```
CSCI49376_Assignment1_LastName/

    Assignment1_Report.pdf
    create_tables.sql
    import_data.sql
    verification_queries.sql
    ER_Diagram.pdf
```

Important Requirements

This assignment is **not simply an exercise in importing a CSV file into a database**.

You are expected to demonstrate the complete relational database design process:

Select Dataset
↓
Examine Raw CSV
↓
Analyze Attributes
↓
Identify Dependencies
↓
Normalize the Dataset
↓
Identify Entities and Relationships
↓
Construct the ER Diagram
↓
Create the Relational Schema
↓
Implement the Database Using SQL
↓
Import the CSV Data
↓
Distribute Data into Normalized Tables
↓
Verify the Database Using SQL

Your final database should preserve the necessary information contained in the original dataset while reducing unnecessary redundancy and establishing appropriate relationships among the data.

Students using different datasets are not expected to produce identical database designs. Your database design should be justified by the structure, dependencies, and relationships that you identify in your selected dataset.